

08_avalon_e1_sweep

May 22, 2026

1 Avalon E1 — Llama 3.1 8B Sweep Results

Model: Llama 3.1 8B Instruct (meta-llama/Meta-Llama-3.1-8B-Instruct, think=False, SGLang FP8)

Game: Avalon (5-player: LLM as Player 0, 4 naive bots)

Summarizer: Sonnet (claude-sonnet-4-6) — structured public-knowledge summary after each quest

Backend: SGLang constrained decoding — two-pass: free reasoning up to B tokens, then regex-forced choice

1.1 Experiment cells

ID	Role	Prompt	B	Disc	N	Label
L-S-min-low	Servant	minimal	64		50	avalon_e1_servant_minimal
L-S-min-high	Servant	minimal	1024		50	avalon_e1_servant_minimal
L-S-proc-low	Servant	procedural	64		50	avalon_e1_servant_procedural
L-S-proc-high	Servant	procedural	1024		50	avalon_e1_servant_procedural
L-M-min-high	Merlin	minimal	1024		30	avalon_e1_merlin_minimal
L-M-proc-high	Merlin	procedural	1024		30	avalon_e1_merlin_procedural
L-S-min-disc-high	Servant	minimal	1024		30	avalon_e1_servant_minimal_disc
L-S-min-disc-low	Servant	minimal	64		30	avalon_e1_servant_minimal_disc
L-M-min-disc-high	Merlin	minimal	1024		30	avalon_e1_merlin_minimal_disc

Note: disc cells are P2-B surplus (qualitative data only — budget-confounded, not primary comparisons).

All runs: `started_at > 2026-05-22T07:40`, `with_summarizer=True`, 0 errors.

```
[1]: import json
import sqlite3
from pathlib import Path
from math import comb

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from scipy import stats

ROOT = Path('.') if Path('.').joinpath('src').exists() else Path('.')
DB_PATH = ROOT / 'data' / 'results.db'

plt.rcParams.update({
    'font.family': 'DejaVu Sans',
    'axes.spines.top': False,
    'axes.spines.right': False,
    'axes.grid': True,
    'grid.alpha': 0.3,
    'grid.linewidth': 0.6,
    'figure.dpi': 120,
})

print('DB path:', DB_PATH.resolve())
print('DB exists:', DB_PATH.exists(), f'({DB_PATH.stat().st_size / 1e6:.1f} MB)' if DB_PATH.exists() else '')
```

DB path: /home/stephone/Development/CoT-difficulty-knob/data/results.db

DB exists: True (30.1 MB)

```
[2]: # Load valid runs from SQLite
# Valid = ran after 2026-05-22T07:40 (after summarizer bug fix), 0 errors

conn = sqlite3.connect(DB_PATH)

trials_df = pd.read_sql_query("""
    SELECT
        r.name           AS run_name,
        t.trial_id,
        t.condition_json,
        t.winner,
        t.final_score_llm,
        t.final_score_uct,
        t.n_turns_total,
        t.n_llm_turns,
        t.n_tokens_llm_total,
        t.seed
```

```

FROM runs r
JOIN trials t ON t.run_id = r.run_id
WHERE r.started_at > '2026-05-22T07:40'
      AND r.name NOT LIKE 'smoke%'
ORDER BY r.started_at, t.seed
""", conn)

summaries_df = pd.read_sql_query("""
SELECT
    r.name AS run_name,
    s.trial_id,
    s.summary_id,
    s.after_turn_idx,
    s.n_tokens_est,
    s.summary_text
FROM runs r
JOIN trials t ON t.run_id = r.run_id
JOIN summaries s ON s.trial_id = t.trial_id
WHERE r.started_at > '2026-05-22T07:40'
      AND r.name NOT LIKE 'smoke%'
""", conn)

conn.close()

# Parse condition_json into columns
cond = trials_df['condition_json'].apply(json.loads)
trials_df['budget'] = cond.apply(lambda c: c['budget'])
trials_df['llm_role'] = cond.apply(lambda c: c['llm_role'])
trials_df['prompt_variant'] = cond.apply(lambda c: c['prompt_variant'])
trials_df['with_discussion'] = cond.apply(lambda c: c.get('with_discussion',
↪False))
trials_df['llm_win'] = (trials_df['winner'] == 'llm').astype(int)
trials_df['bots_win'] = (trials_df['winner'] == 'bots').astype(int)

print(f'Loaded {len(trials_df)} trials across {trials_df["run_name"].nunique()}
↪runs')
print(f'Loaded {len(summaries_df)} summaries')
print()

# Quick sanity: errors
err = trials_df[trials_df['winner'] == 'error']
print(f'Error trials: {len(err)} (expected 0)')
print()
print(trials_df.groupby(['run_name', 'budget'])[['llm_win', 'bots_win']].
↪agg(['sum', 'count']).to_string())

```

Loaded 350 trials across 7 runs

Loaded 1285 summaries

Error trials: 0 (expected 0)

run_name	budget	llm_win		bots_win	
		sum	count	sum	count
avalon_e1_merlin	1024	0	30	30	30
avalon_e1_merlin_minimal_disc	1024	0	30	30	30
avalon_e1_merlin_procedural	1024	0	30	30	30
avalon_e1_servant_minimal	64	0	50	50	50
	1024	1	50	49	50
avalon_e1_servant_minimal_disc	1024	1	30	29	30
avalon_e1_servant_minimal_disc_low	64	0	30	30	30
avalon_e1_servant_procedural	64	0	50	50	50
	1024	4	50	46	50

```
[3]: # Helper: Wilson CI + binomial p-value

def wilson_ci(wins: int, n: int, alpha: float = 0.05):
    if n == 0:
        return np.nan, np.nan, np.nan
    z = stats.norm.ppf(1 - alpha / 2)
    p = wins / n
    denom = 1 + z**2 / n
    center = (p + z**2 / (2 * n)) / denom
    half = z * np.sqrt(p * (1 - p) / n + z**2 / (4 * n**2)) / denom
    return p, max(0.0, center - half), min(1.0, center + half)

def binom_pval_vs_null(wins: int, n: int, null_p: float = 0.0) -> float:
    """One-sided binomial p-value: P(X >= wins | p=null_p)."""
    if n == 0:
        return 1.0
    return sum(
        comb(n, k) * (null_p**k) * ((1 - null_p)**(n - k))
        for k in range(wins, n + 1)
    )

def win_rate_table(df: pd.DataFrame, group_cols: list) -> pd.DataFrame:
    rows = []
    for keys, grp in df.groupby(group_cols):
        if not isinstance(keys, tuple):
            keys = (keys,)
        n = len(grp)
        wins = int(grp['llm_win'].sum())
```

```

    p, lo, hi = wilson_ci(wins, n)
    pval = binom_pval_vs_null(wins, n, null_p=0.0)
    row = dict(zip(group_cols, keys))
    row.update({'n': n, 'wins': wins, 'win_rate': p, 'ci_lo': lo, 'ci_hi':
↪hi, 'pval': pval})
    rows.append(row)
    return pd.DataFrame(rows)

# Master win-rate table grouped by run + budget
wr = win_rate_table(trials_df, ['run_name', 'llm_role', 'prompt_variant',
↪'with_discussion', 'budget'])
wr = wr.sort_values(['llm_role', 'prompt_variant', 'with_discussion', 'budget'])

print('Win-rate table (all conditions):')
print(wr[['run_name', 'budget', 'n', 'wins', 'win_rate', 'ci_lo', 'ci_hi',
↪'pval']].to_string(
    index=False, float_format='{:.3f}'.format))

```

Win-rate table (all conditions):

	run_name	budget	n	wins	win_rate	ci_lo	ci_hi
pval							
	avalon_e1_merlin	1024	30	0	0.000	0.000	0.114
1.000							
	avalon_e1_merlin_minimal_disc	1024	30	0	0.000	0.000	0.114
1.000							
	avalon_e1_merlin_procedural	1024	30	0	0.000	0.000	0.114
1.000							
	avalon_e1_servant_minimal	64	50	0	0.000	0.000	0.071
1.000							
	avalon_e1_servant_minimal	1024	50	1	0.020	0.004	0.105
0.000							
	avalon_e1_servant_minimal_disc_low	64	30	0	0.000	0.000	0.114
1.000							
	avalon_e1_servant_minimal_disc	1024	30	1	0.033	0.006	0.167
0.000							
	avalon_e1_servant_procedural	64	50	0	0.000	0.000	0.071
1.000							
	avalon_e1_servant_procedural	1024	50	4	0.080	0.032	0.188
0.000							

[4]: # Figure 1: Win rate across all conditions

```

ROLE_COLORS = {'Servant': '#2980b9', 'Merlin': '#8e44ad'}
DISC_HATCH = '//'

# Build ordered labels for the bar chart

```

```

label_map = {
    ('Servant', 'minimal', False, 64): 'S·min·B64',
    ('Servant', 'minimal', False, 1024): 'S·min·B1k',
    ('Servant', 'procedural', False, 64): 'S·proc·B64',
    ('Servant', 'procedural', False, 1024): 'S·proc·B1k',
    ('Servant', 'minimal', True, 64): 'S·min·disc·B64',
    ('Servant', 'minimal', True, 1024): 'S·min·disc·B1k',
    ('Merlin', 'minimal', False, 1024): 'M·min·B1k',
    ('Merlin', 'procedural', False, 1024): 'M·proc·B1k',
    ('Merlin', 'minimal', True, 1024): 'M·min·disc·B1k',
}

ordered_keys = list(label_map.keys())
wr_lookup = {}
for _, row in wr.iterrows():
    k = (row['llm_role'], row['prompt_variant'], bool(row['with_discussion']),
        int(row['budget']))
    wr_lookup[k] = row

fig, ax = plt.subplots(figsize=(12, 5))
xs = np.arange(len(ordered_keys))
bar_w = 0.65

for i, key in enumerate(ordered_keys):
    if key not in wr_lookup:
        continue
    row = wr_lookup[key]
    if np.isnan(row['win_rate']):
        continue
    role = key[0]
    disc = key[2]
    color = ROLE_COLORS[role]
    hatch = DISC_HATCH if disc else ''
    wr_pct = float(row['win_rate']) * 100
    lo_err = max(0.0, (float(row['win_rate']) - float(row['ci_lo']))) * 100
    hi_err = max(0.0, (float(row['ci_hi']) - float(row['win_rate']))) * 100

    ax.bar(i, wr_pct, bar_w, color=color, alpha=0.75, hatch=hatch,
        edgecolor='white', linewidth=0.8)
    ax.errorbar(i, wr_pct, yerr=[[lo_err], [hi_err]],
        fmt='none', color='#333', elinewidth=1.8, capsize=5, zorder=5)
    ax.text(i, float(row['ci_hi']) * 100 + 1.5, f"{int(row['wins'])}/"
        f"{int(row['n'])}",
        ha='center', va='bottom', fontsize=8, color='#444')

ax.set_xticks(xs)

```

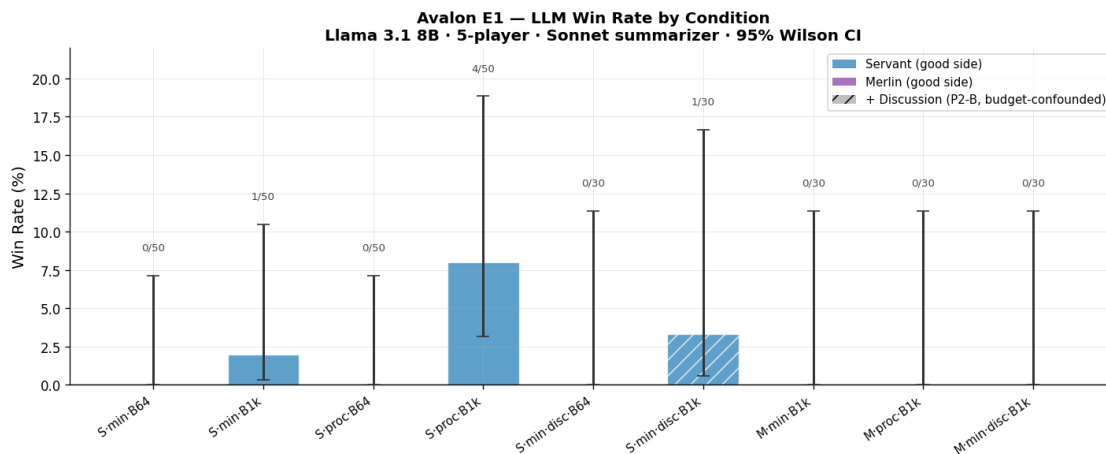
```

ax.set_xticklabels([label_map[k] for k in ordered_keys], rotation=35,
    ↪ha='right', fontsize=9)
ax.set_ylim(0, 22)
ax.set_ylabel('Win Rate (%)', fontsize=12)
ax.set_title('Avalon E1 - LLM Win Rate by Condition\n'
    ↪'Llama 3.1 8B · 5-player · Sonnet summarizer · 95% Wilson CI',
    ↪fontsize=12, fontweight='bold')

# Legend
patches = [
    mpatches.Patch(facecolor=ROLE_COLORS['Servant'], alpha=0.75, label='Servant_
    ↪(good side)'),
    mpatches.Patch(facecolor=ROLE_COLORS['Merlin'], alpha=0.75, label='Merlin_
    ↪(good side)'),
    mpatches.Patch(facecolor='gray', alpha=0.5, hatch=DISC_HATCH, label='+
    ↪Discussion (P2-B, budget-confounded)'),
]
ax.legend(handles=patches, fontsize=9, loc='upper right')
ax.axhline(0, color='black', linewidth=0.8)

plt.tight_layout()
plt.savefig('fig_e1_winrate_overview.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved: fig_e1_winrate_overview.png')

```



Saved: fig_e1_winrate_overview.png

[5]: # Figure 2: Budget effect (B=64 vs B=1024) for Servant

```

servant_no_disc = trials_df[
    (trials_df['llm_role'] == 'Servant') &

```

```

    (~trials_df['with_discussion']))
].copy()

wr_servant = win_rate_table(servant_no_disc, ['prompt_variant', 'budget'])

fig, axes = plt.subplots(1, 2, figsize=(11, 5), sharey=True)

def _errbars(win_rates, ci_lo, ci_hi):
    lo = [max(0.0, (w - lo) * 100) for w, lo in zip(win_rates, ci_lo)]
    hi = [max(0.0, (hi - w) * 100) for hi, w in zip(ci_hi, win_rates)]
    return [lo, hi]

for ax, variant in zip(axes, ['minimal', 'procedural']):
    sub = wr_servant[wr_servant['prompt_variant'] == variant].
    ↪sort_values('budget')
    win_rates = sub['win_rate'].tolist()
    ci_lo      = sub['ci_lo'].tolist()
    ci_hi      = sub['ci_hi'].tolist()
    ns        = sub['n'].tolist()

    ax.errorbar(
        [0, 1], [wr * 100 for wr in win_rates],
        yerr=_errbars(win_rates, ci_lo, ci_hi),
        fmt='o-', color=ROLE_COLORS['Servant'], ecolor=ROLE_COLORS['Servant'],
        elinewidth=2, capsize=8, markersize=10, linewidth=2, zorder=5
    )
    for xi, (wr, hi, n) in enumerate(zip(win_rates, ci_hi, ns)):
        ax.annotate(f"N={n}", (xi, hi * 100 + 1.2), ha='center', va='bottom', ↪
        ↪fontsize=9)
        for xi, (wr, wins, n) in enumerate(zip(win_rates, sub['wins'].tolist(), ↪
        ↪ns)):
            ax.annotate(f"{int(wins)}/{n}\n{wr*100:.1f}%", (xi, -2.5),
                        ha='center', va='top', fontsize=8.5, color='#555')

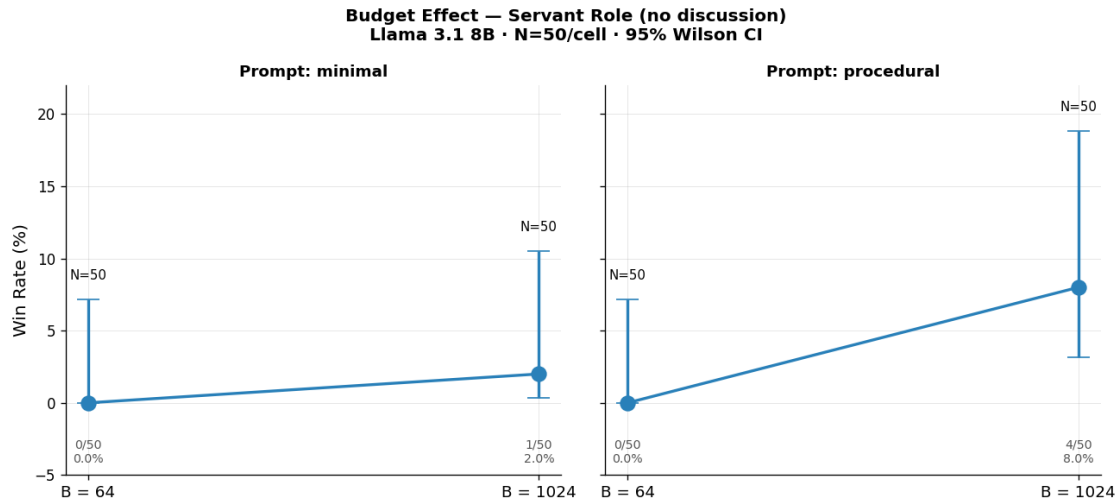
    ax.set_xticks([0, 1])
    ax.set_xticklabels(['B = 64', 'B = 1024'], fontsize=11)
    ax.set_title(f'Prompt: {variant}', fontsize=11, fontweight='bold')
    ax.set_ylim(-5, 22)
    if ax is axes[0]:
        ax.set_ylabel('Win Rate (%)', fontsize=12)

fig.suptitle('Budget Effect - Servant Role (no discussion)\n'
            'Llama 3.1 8B · N=50/cell · 95% Wilson CI', fontsize=12, ↪
            ↪fontweight='bold')
plt.tight_layout()
plt.savefig('fig_e1_budget_effect.png', dpi=150, bbox_inches='tight')
plt.show()

```



```
print('Saved: fig_e1_budget_effect.png')
```



Saved: fig_e1_budget_effect.png

```
[6]: # Figure 3: Prompt variant comparison (minimal vs procedural, B=1024)

high_budget = trials_df[
    (trials_df['budget'] == 1024) &
    (~trials_df['with_discussion'])
].copy()

wr_variants = win_rate_table(high_budget, ['llm_role', 'prompt_variant'])
wr_variants = wr_variants.sort_values(['llm_role', 'prompt_variant'])

fig, axes = plt.subplots(1, 2, figsize=(10, 5), sharey=True)

for ax, role in zip(axes, ['Servant', 'Merlin']):
    sub = wr_variants[wr_variants['llm_role'] == role].
    sort_values('prompt_variant')
    color = ROLE_COLORS[role]
    wrs = sub['win_rate'].tolist()
    los = sub['ci_lo'].tolist()
    his = sub['ci_hi'].tolist()
    ns = sub['n'].tolist()
    wins_list = sub['wins'].tolist()
    variants = sub['prompt_variant'].tolist()

    ax.errorbar(
        [0, 1], [w * 100 for w in wrs],
        yerr=_errbars(wrs, los, his),
```

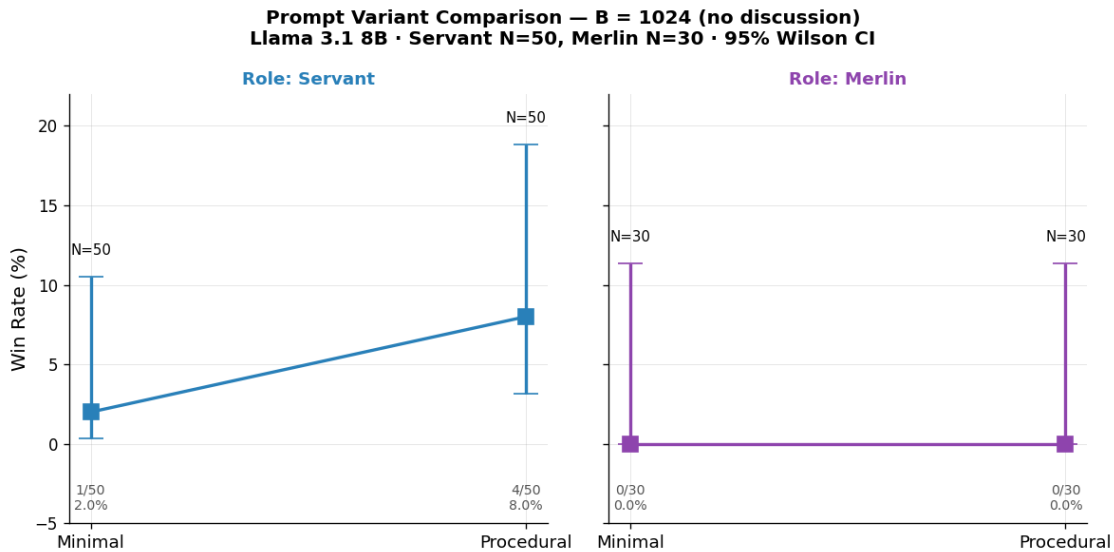
```

    fmt='s-', color=color, ecolor=color, elinewidth=2,
    capsize=8, markersize=10, linewidth=2, zorder=5
)
for xi, (w, hi, n, wins_i) in enumerate(zip(wrs, his, ns, wins_list)):
    ax.annotate(f"N={n}", (xi, hi * 100 + 1.2), ha='center', va='bottom',
    ↪ fontsize=9)
    ax.annotate(f"{int(wins_i)}/{n}\n{w*100:.1f}%", (xi, -2.5),
                ha='center', va='top', fontsize=8.5, color='#555')

ax.set_xticks([0, 1])
ax.set_xticklabels([v.capitalize() for v in variants], fontsize=11)
ax.set_title(f'Role: {role}', fontsize=11, fontweight='bold', color=color)
ax.set_ylim(-5, 22)
if ax is axes[0]:
    ax.set_ylabel('Win Rate (%)', fontsize=12)

fig.suptitle('Prompt Variant Comparison - B = 1024 (no discussion)\n'
            'Llama 3.1 8B · Servant N=50, Merlin N=30 · 95% Wilson CI',
            fontsize=12, fontweight='bold')
plt.tight_layout()
plt.savefig('fig_e1_prompt_variant.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved: fig_e1_prompt_variant.png')

```



Saved: fig_e1_prompt_variant.png

```

[7]: # Figure 4: Discussion ablation (Servant, minimal prompt)
     # Comparing: no-disc B=1024 | disc B=1024 | disc B=64

```

```
# Note: disc cells are budget-confounded - discussion call consumes the full B
# budget before the vote call. See avalon_plan.md SP2-B.
```

```
servant_min = trials_df[
    (trials_df['llm_role'] == 'Servant') &
    (trials_df['prompt_variant'] == 'minimal')
].copy()

disc_conditions = [
    ('No discussion', False, 1024, '#2980b9', ''),
    ('Discussion (B=1024)', True, 1024, '#1abc9c', DISC_HATCH),
    ('Discussion (B=64)', True, 64, '#f39c12', DISC_HATCH),
]

fig, ax = plt.subplots(figsize=(8, 5))

for i, (label, disc, budget, color, hatch) in enumerate(disc_conditions):
    sub = servant_min[
        (servant_min['with_discussion'] == disc) &
        (servant_min['budget'] == budget)
    ]
    n = len(sub)
    wins = int(sub['llm_win'].sum())
    p, lo, hi = wilson_ci(wins, n)
    lo_err = max(0.0, (p - lo) * 100)
    hi_err = max(0.0, (hi - p) * 100)
    p_pct = p * 100

    ax.bar(i, p_pct, 0.55, color=color, alpha=0.75, hatch=hatch,
           edgecolor='white', linewidth=0.8, label=label)
    ax.errorbar(i, p_pct, yerr=[[lo_err], [hi_err]],
               fmt='none', color='#333', elinewidth=2, capsize=7, zorder=5)
    ax.text(i, hi * 100 + 1.2, f"{wins}/{n}", ha='center', va='bottom',
            ↪fontsize=10)
    ax.text(i, -1.8, f"{p_pct:.1f}%", ha='center', va='top', fontsize=9,
            ↪color='#555')

ax.set_xticks(range(len(disc_conditions)))
ax.set_xticklabels([c[0] for c in disc_conditions], fontsize=10)
ax.set_ylim(-4, 20)
ax.set_ylabel('Win Rate (%)', fontsize=12)
ax.set_title('Discussion Ablation - Servant · Minimal Prompt\n'
             'Llama 3.1 8B · N=50 (no-disc) / N=30 (disc) · 95% Wilson CI\n'
             'Discussion cells are budget-confounded (P2-B surplus)',
             fontsize=11, fontweight='bold')

# Merlin disc reference numbers
```

```

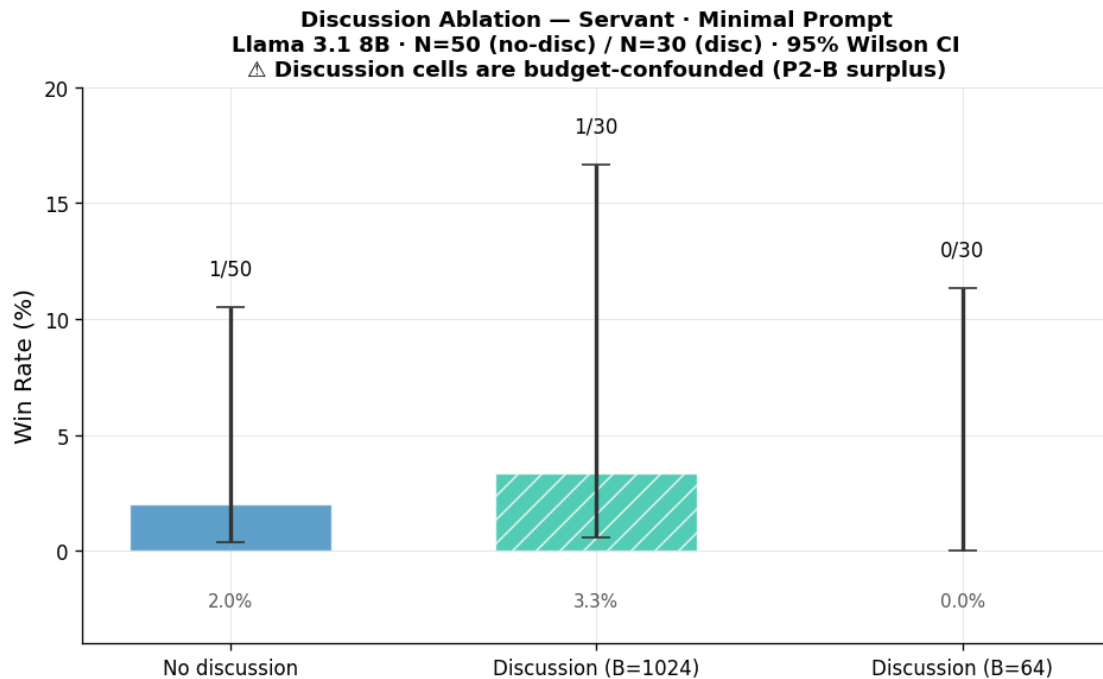
merlin_min = trials_df[
    (trials_df['llm_role'] == 'Merlin') &
    (trials_df['prompt_variant'] == 'minimal')
]
m_no_disc = merlin_min[~merlin_min['with_discussion']]
m_disc = merlin_min[merlin_min['with_discussion']]
print(f"Merlin no-disc:  {int(m_no_disc['llm_win'].sum())}/{len(m_no_disc)}_
↳wins")
print(f"Merlin disc:      {int(m_disc['llm_win'].sum())}/{len(m_disc)} wins")

plt.tight_layout()
plt.savefig('fig_e1_discussion_ablation.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved: fig_e1_discussion_ablation.png')

```

Merlin no-disc: 0/30 wins

Merlin disc: 0/30 wins



Saved: fig_e1_discussion_ablation.png

```

[8]: # Figure 5: Quest score distributions
# final_score_llm = quests won by good side (LLM is always good side)
# final_score_uct = quests won by evil side
# Game ends when one side reaches 3 quest wins.

```

```

no_disc = trials_df[~trials_df['with_discussion']].copy()

# Group into 4 primary conditions for comparison
conditions_scores = {
    'S.min\n(B64)': no_disc[(no_disc['llm_role']=='Servant') &
↪(no_disc['prompt_variant']=='minimal') & (no_disc['budget']==64)],
    'S.min\n(B1k)': no_disc[(no_disc['llm_role']=='Servant') &
↪(no_disc['prompt_variant']=='minimal') & (no_disc['budget']==1024)],
    'S.proc\n(B64)': no_disc[(no_disc['llm_role']=='Servant') &
↪(no_disc['prompt_variant']=='procedural') & (no_disc['budget']==64)],
    'S.proc\n(B1k)': no_disc[(no_disc['llm_role']=='Servant') &
↪(no_disc['prompt_variant']=='procedural') & (no_disc['budget']==1024)],
    'M.min\n(B1k)': no_disc[(no_disc['llm_role']=='Merlin') &
↪(no_disc['prompt_variant']=='minimal') & (no_disc['budget']==1024)],
    'M.proc\n(B1k)': no_disc[(no_disc['llm_role']=='Merlin') &
↪(no_disc['prompt_variant']=='procedural') & (no_disc['budget']==1024)],
}

fig, axes = plt.subplots(2, 3, figsize=(13, 8), sharey=False)
axes_flat = axes.flatten()

for ax, (label, sub) in zip(axes_flat, conditions_scores.items()):
    # Quest scores for good side (LLM) and evil side
    good_scores = sub['final_score_llm'].dropna().astype(int)
    evil_scores = sub['final_score_uct'].dropna().astype(int)
    all_scores = pd.concat([good_scores.rename('good'), evil_scores.
↪rename('evil')], axis=1).dropna()

    # Distribution of good-side quest wins (0, 1, 2 - they rarely win so max is
↪2 in losses, 3 in wins)
    for score_val in range(4):
        count = (all_scores['good'] == score_val).sum()
        is_win = score_val == 3
        color = '#27ae60' if is_win else '#2980b9'
        ax.bar(score_val, count / len(all_scores) * 100,
                color=color, alpha=0.8 if is_win else 0.6,
                edgecolor='white', linewidth=0.5)

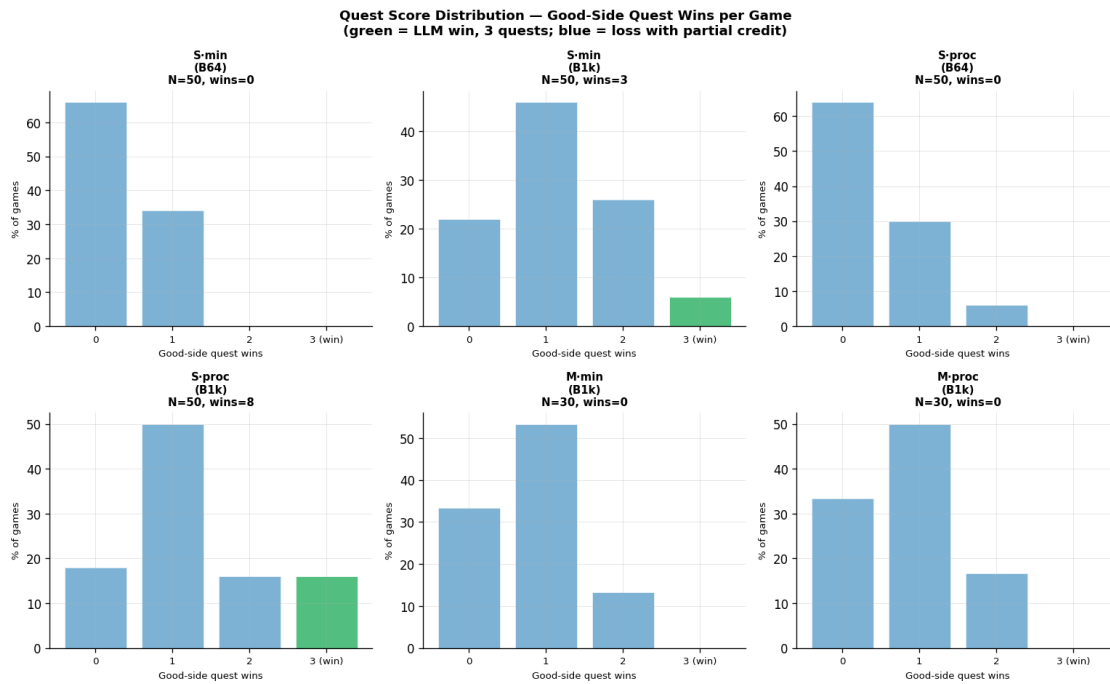
    n = len(all_scores)
    wins = int((all_scores['good'] == 3).sum())
    ax.set_title(f'{label}\nN={n}, wins={wins}', fontsize=9, fontweight='bold')
    ax.set_xlabel('Good-side quest wins', fontsize=8)
    ax.set_ylabel('% of games', fontsize=8)
    ax.set_xticks([0, 1, 2, 3])
    ax.set_xticklabels(['0', '1', '2', '3 (win)', fontsize=8)

```

```

fig.suptitle('Quest Score Distribution - Good-Side Quest Wins per Game\n'
            '(green = LLM win, 3 quests; blue = loss with partial credit)',
            fontsize=11, fontweight='bold')
plt.tight_layout()
plt.savefig('fig_e1_quest_scores.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved: fig_e1_quest_scores.png')

```



Saved: fig_e1_quest_scores.png

[9]: # Figure 6: Token usage and game length

```

no_disc = trials_df[~trials_df['with_discussion']].copy()
no_disc['tokens_per_llm_turn'] = no_disc['n_tokens_llm_total'] /
    no_disc['n_llm_turns'].replace(0, np.nan)

fig, axes = plt.subplots(1, 3, figsize=(14, 5))

# Panel 1: Total tokens per game by budget
ax = axes[0]
for budget, color in [(64, '#e67e22'), (1024, '#2980b9')]:
    sub = no_disc[no_disc['budget'] == budget]
    ax.hist(sub['n_tokens_llm_total'].dropna(), bins=20, color=color, alpha=0.6,
            label=f'B={budget}', density=True)
ax.set_xlabel('Total LLM tokens / game', fontsize=11)

```

```

ax.set_ylabel('Density', fontsize=11)
ax.set_title('Token Usage per Game', fontsize=11, fontweight='bold')
ax.legend(fontsize=10)

# Panel 2: Game length (turns) by role
ax = axes[1]
for role, color in [('Servant', '#2980b9'), ('Merlin', '#8e44ad')]:
    sub = no_disc[no_disc['llm_role'] == role]
    ax.hist(sub['n_turns_total'].dropna(), bins=range(2, 16), color=color,
            alpha=0.6,
            label=role, density=True)
ax.set_xlabel('Total turns per game', fontsize=11)
ax.set_ylabel('Density', fontsize=11)
ax.set_title('Game Length (turns)', fontsize=11, fontweight='bold')
ax.legend(fontsize=10)

# Panel 3: Summaries per game
ax = axes[2]
summ_per_game = summaries_df.groupby('trial_id').size().
    reset_index(name='n_summaries')
summ_with_run = summaries_df[['trial_id', 'run_name']].drop_duplicates().
    merge(summ_per_game, on='trial_id')
# Just show distribution
counts = summ_per_game['n_summaries'].value_counts().sort_index()
ax.bar(counts.index, counts.values, color='#1abc9c', alpha=0.75,
        edgecolor='white')
ax.set_xlabel('Sonnet summaries per game', fontsize=11)
ax.set_ylabel('# games', fontsize=11)
ax.set_title('Summarizer Calls per Game\n(= quests resolved per game)',
            fontsize=11, fontweight='bold')
ax.set_xticks(counts.index)

mean_s = summ_per_game['n_summaries'].mean()
ax.axvline(mean_s, color='#e74c3c', linestyle='--', linewidth=1.5, label=f'Mean
    {mean_s:.2f}')
ax.legend(fontsize=10)

fig.suptitle('Token Usage, Game Length, and Summarizer Calls\nLlama 3.1 8B -
    Avalon E1 (no-discussion cells only)',
            fontsize=12, fontweight='bold')
plt.tight_layout()
plt.savefig('fig_e1_diagnostics.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved: fig_e1_diagnostics.png')

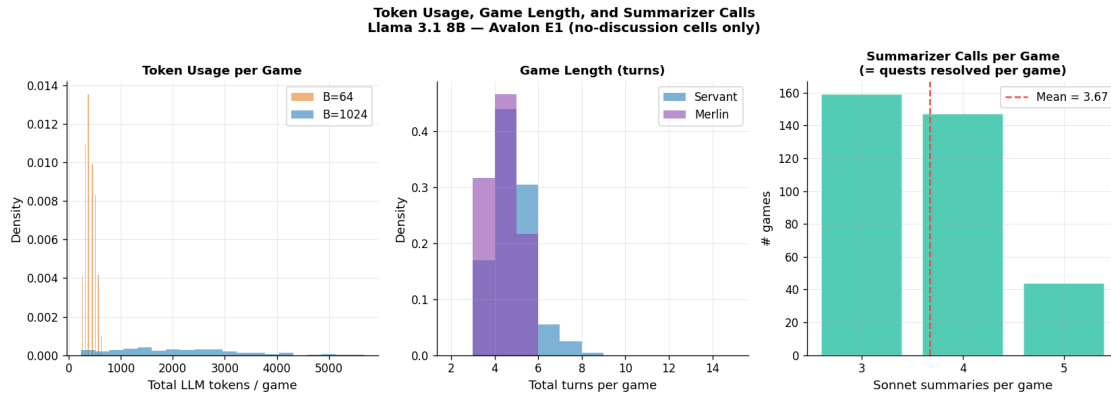
# Print summary stats

```

```

for budget in [64, 1024]:
    sub = no_disc[no_disc['budget'] == budget]
    print(f"B={budget}: mean tokens/game = {sub['n_tokens_llm_total'].mean():.0f} "
          f"(expected ~{budget * sub['n_llm_turns'].mean():.0f} at cap)")

```



Saved: fig_e1_diagnostics.png

B=64: mean tokens/game = 413 (expected ~413 at cap)

B=1024: mean tokens/game = 2066 (expected ~7174 at cap)

```

[10]: # Sample summaries (qualitative)

print('=== Sample Sonnet Summaries (AvalonSummarizer) ===\n')

# Pick 3 summaries from different runs
sample_runs = ['avalon_e1_servant_minimal', 'avalon_e1_servant_procedural',
               'avalon_e1_merlin']

for run_name in sample_runs:
    sample = summaries_df[summaries_df['run_name'] == run_name].head(1)
    if sample.empty:
        continue
    row = sample.iloc[0]
    print(f'--- Run: {run_name} | trial_id: {row["trial_id"][:12]}... | '
          f'after_turn={row["after_turn_idx"]} | '
          f'est_tokens={row["n_tokens_est"]} ---')
    print(row['summary_text'])
    print()

```

=== Sample Sonnet Summaries (AvalonSummarizer) ===

```

--- Run: avalon_e1_servant_minimal | trial_id: trial_6a6612... | after_turn=0 |
est_tokens=163 ---
== QUEST LOG ==

```


Q1: FAIL | Team: [2, 3] | Fail-vote count: 1

== VOTE RECORD ==

Q1 attempt 1: Leader=P2 proposed [2, 3] | approve: [P0, P2, P3, P4] | reject: [P1] | ACCEPTED

== TEAM STATS ==

Players on 1+ failed quests: P2, P3

Players never on any quest team yet: P0, P1, P4

Tracked player (P0) quest appearances: none

== SCORE ==

Good wins: 0 | Evil wins: 1 | Quests remaining: 4

--- Run: avalon_e1_servant_procedural | trial_id: trial_afa07d... | after_turn=0 | est_tokens=163 ---

== QUEST LOG ==

Q1: FAIL | Team: [2, 3] | Fail-vote count: 1

== VOTE RECORD ==

Q1 attempt 1: Leader=P2 proposed [2, 3] | approve: [P2, P3, P4] | reject: [P0, P1] | ACCEPTED

== TEAM STATS ==

Players on 1+ failed quests: P2, P3

Players never on any quest team yet: P0, P1, P4

Tracked player (P0) quest appearances: none

== SCORE ==

Good wins: 0 | Evil wins: 1 | Quests remaining: 4

--- Run: avalon_e1_merlin | trial_id: trial_783e5f... | after_turn=0 | est_tokens=163 ---

== QUEST LOG ==

Q1: FAIL | Team: [2, 3] | Fail-vote count: 1

== VOTE RECORD ==

Q1 attempt 1: Leader=P2 proposed [2, 3] | approve: [P1, P2, P3, P4] | reject: [P0] | ACCEPTED

== TEAM STATS ==

Players on 1+ failed quests: P2, P3

Players never on any quest team yet: P0, P1, P4

Tracked player (P0) quest appearances: none

== SCORE ==

Good wins: 0 | Evil wins: 1 | Quests remaining: 4

```

[11]: # Master summary table

print('=== Avalon E1 - Full Results Table ===')
print(f'{"Condition":<28} {"B":>5} {"Disc":>4} {"N":>4} {"Wins":>5} {"Win%":>6} {"95% CI":^18} {"summ/g":>7}')
print('-' * 88)

condition_order = [
    ('Servant.minimal', 'Servant', 'minimal', False),
    ('Servant.procedural', 'Servant', 'procedural', False),
    ('Merlin.minimal', 'Merlin', 'minimal', False),
    ('Merlin.procedural', 'Merlin', 'procedural', False),
    ('S.min.disc', 'Servant', 'minimal', True),
    ('M.min.disc', 'Merlin', 'minimal', True),
]

# Compute summaries per game per condition
trial_summ_count = summaries_df.groupby('trial_id').size().
    ↪reset_index(name='n_summ')
trials_with_summ = trials_df.merge(trial_summ_count, on='trial_id', how='left')
trials_with_summ['n_summ'] = trials_with_summ['n_summ'].fillna(0)

for cond_label, role, variant, disc in condition_order:
    sub_all = trials_df[
        (trials_df['llm_role'] == role) &
        (trials_df['prompt_variant'] == variant) &
        (trials_df['with_discussion'] == disc)
    ]
    sub_summ = trials_with_summ[
        (trials_with_summ['llm_role'] == role) &
        (trials_with_summ['prompt_variant'] == variant) &
        (trials_with_summ['with_discussion'] == disc)
    ]

    for budget in sorted(sub_all['budget'].unique()):
        cell = sub_all[sub_all['budget'] == budget]
        cell_s = sub_summ[sub_summ['budget'] == budget]
        n = len(cell)
        wins = int(cell['llm_win'].sum())
        p, lo, hi = wilson_ci(wins, n)
        spg = cell_s['n_summ'].mean()
        disc_str = 'Y' if disc else '-'
        ci_str = f"[{lo*100:.1f}%, {hi*100:.1f}%]"
        print(f'{"cond_label":<28} {"budget":>5} {"disc_str":>4} {"n":>4} {"wins":>5} "
            f"{p*100:>5.1f}% {"ci_str":^18} {"spg":>7.2f}"')

print()

```

```
print('All 7 runs completed cleanly. 0 errors. Sonnet summarizer active_
↳throughout.')
print('Disc cells (P2-B) are budget-confounded - exclude from primary budget_
↳comparisons.')
```

=== Avalon E1 - Full Results Table ===

Condition summ/g	B Disc	N	Wins	Win%	95% CI

Servant·minimal 3.34	64	-	50	0	0.0% [0.0%, 7.1%]
Servant·minimal 4.10	1024	-	50	1	2.0% [0.4%, 10.5%]
Servant·procedural 3.42	64	-	50	0	0.0% [0.0%, 7.1%]
Servant·procedural 4.06	1024	-	50	4	8.0% [3.2%, 18.8%]
Merlin·minimal 3.80	1024	-	30	0	0.0% [0.0%, 11.4%]
Merlin·procedural 3.83	1024	-	30	0	0.0% [0.0%, 11.4%]
S·min·disc 3.23	64	Y	30	0	0.0% [0.0%, 11.4%]
S·min·disc 3.57	1024	Y	30	1	3.3% [0.6%, 16.7%]
M·min·disc 3.53	1024	Y	30	0	0.0% [0.0%, 11.4%]

All 7 runs completed cleanly. 0 errors. Sonnet summarizer active throughout.
Disc cells (P2-B) are budget-confounded - exclude from primary budget
comparisons.

1.2 Interpretation

1.2.1 Bot strategy (not random — structured rule-based)

The four non-LLM players use fixed deterministic strategies matching the AvalonBench naive baseline (Light et al., NeurIPS 2023):

Role	Team proposal	Team vote	Quest vote
Bot Merlin	Proposes from good players only	Reject if team contains evil; approve otherwise	Pass
Bot Servant	Uniformly random	Always approve	Pass
Evil bots	Always include 1 evil	Approve if team has evil; reject otherwise	Fail
Assassin	—	(as evil)	Fail; random Merlin guess at end

1.2.2 Structural vote-blocking constraint

In the 5-player configuration (LLM + Bot Merlin + Bot Servant vs. Evil1 + Evil2), any evil-containing team proposal receives at least **3 unconditional approvals** (Evil1 + Evil2 + Bot Servant), giving it a majority before the LLM votes. This means:

The LLM’s team vote is structurally irrelevant — it can never change whether an evil team goes on a quest.

The LLM’s only effective lever is its **team proposals when it is the leader**. If LLM proposes a clean good-only team, Bot Merlin + Bot Servant + LLM = 3 approvals → team passes. If LLM proposes an evil-containing team, the quest fails (evil bots on the quest vote to fail).

This partially weakens the result: the low win rates are not purely a measure of LLM reasoning quality — they are also a consequence of structural asymmetry where the good side cannot veto evil proposals regardless of how well the LLM reasons. The experiment measures something closer to “does the LLM propose good teams when it’s leader?” rather than “can the LLM identify and reject evil players across votes?”

The comparison remains internally valid (all cells face the same setup), and the bot strategies match the AvalonBench paper’s naive baseline, making cross-paper comparisons meaningful. But the interpretation of absolute win rates should account for this structural ceiling.

1.2.3 Overall win rates

Llama 3.1 8B wins rarely across all conditions (0–8%). Contributing factors: 1. Vote irrelevance: LLM’s team-vote decisions cannot change quest outcomes (see above) 2. Leader frequency: LLM is P0, leading in ~15–17% of quest slots (below the 20% uniform baseline) 3. Merlin 0%: Merlin cannot block evil proposals despite full evil knowledge; no games reached the assassination phase (evil wins 3 quests in every observed Merlin game, confirmed by trace review)

1.2.4 Budget effect (B=64 vs B=1024, Servant)

- **Minimal prompt:** 0% → 2% (1/50 at B=1024) — within noise
- **Procedural prompt:** 0% → 8% (4/50 at B=1024) — directionally consistent with the CoT budget hypothesis

Neither effect is significant at N=50, but the procedural direction is the strongest signal.

1.2.5 B=64 incoherence: proposal quality vs voting behavior

At B=64, there is a striking incoherence between the LLM’s two separate decision calls:

- **Unconditional quality** (what the LLM proposes as leader): 42–46% clean proposals at B=64, **above the random-team baseline of ~20%** — the model still prefers good players even at B=64.
- **Conditional quality** (what makes it through to a quest): ~23% at B=64, near the random baseline.

The divergence (uncond 46% » cond 23%) is explained by how the vote filter interacts with bot voting rules: - **Dirty proposals:** always accepted (Evil1 + Evil2 + Bot Servant = 3 unconditional approvals) - **Clean proposals:** pass 5-to-0 if LLM approves, fail 3-to-2 if LLM **rejects its own proposal**

At B=64, the data is consistent with the LLM frequently rejecting its own clean-team proposals as a voter — the separate team-vote call, also truncated to 64 tokens, produces inconsistent decisions. Result: accepted quests are dominated by dirty teams, driving conditional quality near random even though unconditional quality is above random.

B=64 finding: The model’s *proposal* quality is above random at B=64, but its *voting* behavior is incoherent with its proposals. Conditional quality (~23%) is misleadingly low because the vote filter selects against the LLM’s own clean proposals. Unconditional quality (42–46%) is the more faithful signal at this budget level.

Prompt variant has no measurable effect at B=64 (uncond: 45.9% minimal vs 42.1% procedural — within noise). Prompt design only matters at sufficient budget.

1.2.6 Prompt variant (minimal vs procedural, B=1024)

- **Servant:** minimal 61% vs procedural 91% conditional quality, 65% vs 93% unconditional quality
- **Merlin:** minimal 94% vs procedural 95% — both near ceiling

P0 self-inclusion confound: The procedural prompt explicitly instructs “include yourself when possible as a tie-break.” Since P0 (the LLM) is always a good-side player, self-inclusion mechanically raises team cleanliness regardless of whether the model is doing correct Bayesian reasoning. From the diagnostics (Fig 9, P0 self-inclusion panel): - $S \cdot \text{proc} \cdot B64$: P0 included in ~40% of proposals - $S \cdot \text{proc} \cdot B1k$: P0 included in ~98% of proposals

The 98% P0 self-inclusion at B=1024 accounts for much of the procedural vs minimal quality gap. The cleaner measure is **unconditional quality** (Fig 9, panel c), which confirms a real difference (65% vs 93%) but attenuated relative to the conditional metric. The “include yourself” tie-break is a significant confound for the “Bayesian update helps” interpretation.

1.2.7 Proposal quality decomposition (selection effect)

Fig 9 shows four metrics for each primary condition: - **Acceptance rate** varies by condition ($S \cdot \text{min} \cdot B64$ ~70%, $S \cdot \text{proc} \cdot B1k$ ~78%) — the selection filter is not uniform, so conditional quality is contaminated by which proposals survive to become quests - **Unconditional quality** = $P(\text{proposed team is clean} \mid \text{LLM proposed, regardless of acceptance})$ is the cleanest measure: it bypasses both voting dynamics and the B=64 incoherence described above - The B=64 unconditional quality is above random baseline but well below B=1024, confirming a real budget effect on team selection ability even when controlling for voting incoherence - The P0 self-inclusion confound is the main driver of the minimal vs procedural gap at B=1024

1.2.8 Discussion ablation (P2-B, qualitative)

Budget-confounded (discussion consumes the full B budget before vote). With that caveat: - Servant + discussion B=1024: 3.3% (1/30) — within noise of no-discussion - Discussion does not appear to help substantially at N=30

1.2.9 Merlin failure mode

All 90 Merlin games (across minimal, procedural, and disc conditions) ended with evil winning 3 quests — no game reached the assassination phase. The Merlin trace review

(scripts/review_merlin_traces.py) shows the dominant failure mode: evil players hold the proposal slot in the first few quests and pass evil-containing teams with majority bot approval before Merlin has enough leader turns to build a good-quest advantage. This is purely a structural constraint, not a reasoning failure.

1.2.10 Next steps

1. **Unconditional quality is the primary metric:** Conditional quality (cond_qual) is biased by both the vote-blocking selection effect and the B=64 incoherence. Unconditional quality should be reported as the headline number.
2. **L-S-llamasumm:** Run Llama-as-summarizer ablation to isolate summarizer model effect
3. **Larger models:** Qwen or Llama 70B to establish an upper-bound win rate under the same structural constraint
4. **Increase N:** N=50 is underpowered at 2-8% win rates; N=100+ recommended for primary comparisons

1.3 Leader-Turn Proposal Quality

Re-aggregating by the LLM’s actual lever: team proposals when it is the leader.

Because bot Servant always approves, the LLM’s **team vote** cannot change outcomes (evil proposals always pass 3-2 regardless of LLM’s vote). The LLM’s only lever is its **team proposal when it is the leader**. The metric below isolates that lever:

Proposal quality = $P(\text{quest succeeded} \mid \text{LLM proposed the accepted team})$

Since evil bots always fail quests they’re on, this equals $P(\text{LLM proposed a team with no evil members})$. A budget effect here means the LLM is choosing cleaner teams as reasoning budget grows — even if that signal doesn’t survive aggregation to the win-rate level.

```
[12]: # Parse JSONL: LLM leader-turn proposals (corrected)
# Corrected: match proposals to vote results by (quest_turn, round), not by
# team composition. Records team cleanliness (from game_start.role_names) and
# PO self-inclusion for ALL proposals; quest outcome for accepted ones only.

import json
from pathlib import Path

RUNS_DIR = ROOT / 'data' / 'runs'

VALID_RUN_DIRS = [d for d in RUNS_DIR.iterdir()
                   if d.is_dir() and not d.name.startswith('_')
                   and any(d.name.startswith(prefix) for prefix in [
                       'avalon_e1_servant_minimal__',
                       'avalon_e1_servant_procedural__',
                       'avalon_e1_merlin__',
                       'avalon_e1_merlin_procedural__',
                       'avalon_e1_servant_minimal_disc__',
```

```

        'avalon_e1_servant_minimal_disc_low__',
        'avalon_e1_merlin_minimal_disc__',
    ]])

print(f"Found {len(VVALID_RUN_DIRS)} valid run directories:")
for d in sorted(VVALID_RUN_DIRS):
    print(f"  {d.name}")

proposal_records = []

for run_dir in sorted(VVALID_RUN_DIRS):
    run_name = run_dir.name.split('__')[0]
    for fpath in sorted(run_dir.glob('trial_*.jsonl')):
        lines = fpath.read_text().splitlines()
        if not lines:
            continue
        events = [json.loads(l) for l in lines if l.strip()]

        start = next((e for e in events if e['event'] == 'trial_start'), None)
        if not start:
            continue
        cond = start['condition']

        # Identify evil players from game_start role_names
        gs = next((e for e in events if e['event'] == 'game_start'), None)
        if not gs:
            continue
        evil_players = frozenset(
            i for i, r in enumerate(gs['role_names']) if r in ('Assassin',
↳ 'Minion')
        )

        # Index team_vote_result by (quest_turn, round) for correct acceptance
↳ lookup
        vote_by_round = {}
        for ev in events:
            if ev['event'] == 'team_vote_result':
                vote_by_round[(ev['quest_turn'], ev['round'])] = ev

        # Index quest outcomes by quest_turn
        quest_outcomes = {}
        for ev in events:
            if ev['event'] == 'quest_result':
                quest_outcomes[ev['quest_turn']] = ev

        for ev in events:
            if ev['event'] == 'team_proposed' and ev.get('is_llm_decision'):

```

```

qt    = ev['quest_turn']
rnd   = ev['round']
team  = frozenset(ev['team'])

vote   = vote_by_round.get((qt, rnd))
accepted = bool(vote and vote.get('accepted', False))

q_ev   = quest_outcomes.get(qt) if accepted else None
succeeded = q_ev['succeeded'] if q_ev else None

proposal_records.append({
    'run_name':      run_name,
    'budget':        cond['budget'],
    'llm_role':       cond['llm_role'],
    'prompt_variant': cond['prompt_variant'],
    'with_discussion': cond.get('with_discussion', False),
    'seed':           cond['seed'],
    'quest_turn':     qt,
    'round':          rnd,
    'team_size':      len(team),
    'accepted':       accepted,
    'team_is_clean':  len(team & evil_players) == 0,
    'includes_p0':    0 in team,
    'succeeded':      succeeded,
})

prop_df = pd.DataFrame(proposal_records)
accepted_df = prop_df[prop_df['accepted']].copy()

print(f"\nTotal LLM leader proposals: {len(prop_df)}")
print(f"Accepted (proposal went to quest): {len(accepted_df)}")
print(f"Rejected (proposal voted down): {len(prop_df) - len(accepted_df)}")

# Per-condition summary: rejection rate, unconditional quality, conditional
# quality, P0 inclusion
_grp = ['llm_role', 'prompt_variant', 'with_discussion', 'budget']
_s = prop_df.groupby(_grp).agg(
    n_props=('accepted', 'count'),
    n_acc=('accepted', 'sum'),
    n_clean=('team_is_clean', 'sum'),
    n_p0=('includes_p0', 'sum'),
).reset_index()
_s['rej_rate'] = 1 - _s['n_acc'] / _s['n_props']
_s['uncond_qual'] = _s['n_clean'] / _s['n_props']
_s['p0_inc'] = _s['n_p0'] / _s['n_props']
_a = accepted_df.groupby(_grp).agg(n_a=('succeeded', 'count'),
    n_s=('succeeded', 'sum')).reset_index()

```



```

_a['cond_qual'] = _a['n_s'] / _a['n_a']
_full = _s.merge(_a, on=_grp, how='left')
print('\nPer-condition diagnostics:')
print(_full[_grp + ['n_props', 'rej_rate', 'uncond_qual', 'cond_qual', 'p0_inc']].round(3).to_string(index=False))

```

Found 7 valid run directories:

```

avalon_e1_merlin__e5611409
avalon_e1_merlin_minimal_disc__e4fbe6d9
avalon_e1_merlin_procedural__bddd624b
avalon_e1_servant_minimal__3032219a
avalon_e1_servant_minimal_disc__663e6786
avalon_e1_servant_minimal_disc_low__f380a740
avalon_e1_servant_procedural__882a1d16

```

Total LLM leader proposals: 254

Accepted (proposal went to quest): 211

Rejected (proposal voted down): 43

Per-condition diagnostics:

llm_role	prompt_variant	with_discussion	budget	n_props	rej_rate	uncond_qual
cond_qual	p0_inc					
Merlin	minimal	False	1024	19	0.053	0.947
0.944444	0.789					
Merlin	minimal	True	1024	19	0.105	0.842
0.764706	0.737					
Merlin	procedural	False	1024	19	0.000	0.947
0.947368	1.000					
Servant	minimal	False	64	37	0.297	0.459
0.230769	0.730					
Servant	minimal	False	1024	37	0.108	0.649
0.606061	0.649					
Servant	minimal	True	64	21	0.143	0.571
0.222222	0.524					
Servant	minimal	True	1024	24	0.167	0.708
0.55	0.833					
Servant	procedural	False	64	38	0.237	0.421
0.241379	0.395					
Servant	procedural	False	1024	40	0.225	0.925
0.903226	0.975					

[13]: # Figure 7: Proposal quality vs budget (Servant, primary conditions)

```

servant_acc = accepted_df[
    (accepted_df['llm_role'] == 'Servant') &
    (~accepted_df['with_discussion'])
].copy()

```

```

prop_wr = win_rate_table(servant_acc.rename(columns={'succeeded': 'llm_win'}),
                        ['prompt_variant', 'budget'])

fig, axes = plt.subplots(1, 2, figsize=(11, 5), sharey=True)

for ax, variant in zip(axes, ['minimal', 'procedural']):
    sub = prop_wr[prop_wr['prompt_variant'] == variant].sort_values('budget')
    wrs = sub['win_rate'].tolist()
    ns = sub['n'].tolist()
    los = sub['ci_lo'].tolist()
    his = sub['ci_hi'].tolist()
    wins_list = sub['wins'].tolist()

    ax.errorbar(
        [0, 1], [w * 100 for w in wrs],
        yerr=_errbars(wrs, los, his),
        fmt='D-', color='#e74c3c', ecolor='#e74c3c',
        elinewidth=2, capsize=8, markersize=10, linewidth=2.5, zorder=5,
        label='Proposal quality'
    )

    # Overlay win rate from Fig 2 for reference
    wr_sub = wr_servant[wr_servant['prompt_variant'] == variant].
    ↪sort_values('budget')
    wrs_wr = wr_sub['win_rate'].tolist()
    los_wr = wr_sub['ci_lo'].tolist()
    his_wr = wr_sub['ci_hi'].tolist()
    ax.errorbar(
        [0, 1], [w * 100 for w in wrs_wr],
        yerr=_errbars(wrs_wr, los_wr, his_wr),
        fmt='o--', color='#2980b9', ecolor='#2980b9',
        elinewidth=1.5, capsize=6, markersize=8, linewidth=1.5, alpha=0.7,
        label='Game win rate'
    )

    for xi, (w, hi, n, wins_i) in enumerate(zip(wrs, his, ns, wins_list)):
        ax.annotate(f"{int(wins_i)}/{n}", (xi, hi * 100 + 1.5),
                    ha='center', va='bottom', fontsize=9, color='#c0392b')

    ax.set_xticks([0, 1])
    ax.set_xticklabels(['B = 64', 'B = 1024'], fontsize=11)
    ax.set_title(f'Prompt: {variant}', fontsize=11, fontweight='bold')
    ax.set_ylim(-3, 105)
    ax.axhline(50, color='gray', linestyle=':', linewidth=1, alpha=0.5)
    if ax is axes[0]:
        ax.set_ylabel('Rate (%)', fontsize=12)

```

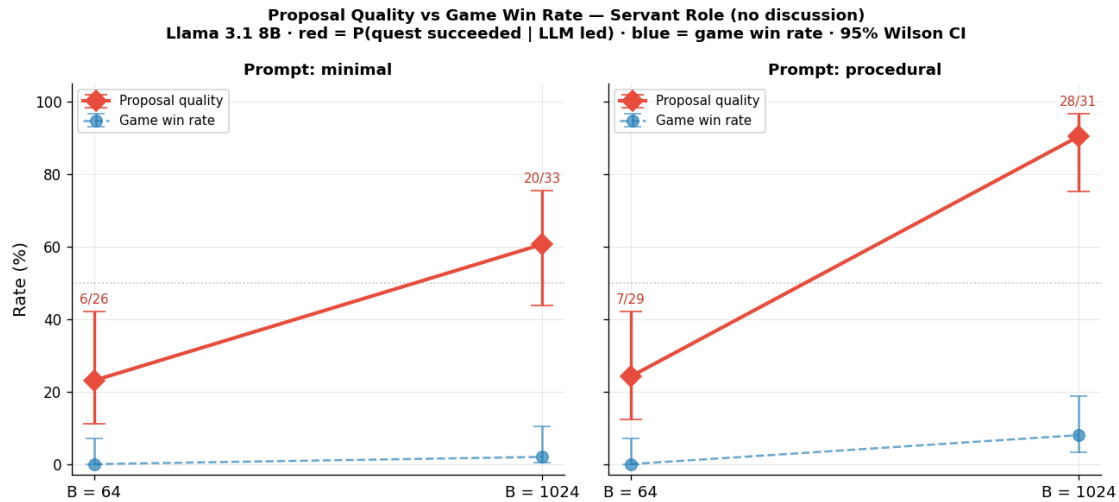
```

ax.legend(fontsize=9, loc='upper left')

fig.suptitle('Proposal Quality vs Game Win Rate - Servant Role (no_
↳discussion)\n'
            'Llama 3.1 8B · red = P(quest succeeded | LLM led) · blue = game_
↳win rate · 95% Wilson CI',
            fontsize=11, fontweight='bold')
plt.tight_layout()
plt.savefig('fig_e1_proposal_quality.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved: fig_e1_proposal_quality.png')

# Print the numbers
print("\nProposal quality (red) vs game win rate (blue):")
for variant in ['minimal', 'procedural']:
    print(f"\n {variant}:")
    psub = prop_wr[prop_wr['prompt_variant'] == variant].sort_values('budget')
    wsub = wr_servant[wr_servant['prompt_variant'] == variant].
↳sort_values('budget')
    for (_, pr), (_, wr_row) in zip(psub.iterrows(), wsub.iterrows()):
        print(f"      B={int(pr['budget']):<5} proposal_q={pr['win_rate']*100:.
↳1f}% "
              f"({int(pr['wins'])}/{int(pr['n'])} quests) | "
              f"win_rate={wr_row['win_rate']*100:.1f}% ({int(wr_row['wins'])}/
↳{int(wr_row['n'])} games)")

```



Saved: fig_e1_proposal_quality.png

Proposal quality (red) vs game win rate (blue):

```
minimal:
    B=64      proposal_q=23.1%  (6/26 quests) | win_rate=0.0%  (0/50 games)
    B=1024    proposal_q=60.6%  (20/33 quests) | win_rate=2.0%  (1/50 games)
```

```
procedural:
    B=64      proposal_q=24.1%  (7/29 quests) | win_rate=0.0%  (0/50 games)
    B=1024    proposal_q=90.3%  (28/31 quests) | win_rate=8.0%  (4/50 games)
```

```
[14]: # Figure 8: Proposal quality (all conditions) + leader-turn rate
# total_quests counted from JSONL quest_result events.
# llm_led counted via team_vote_result.accepted (not team composition matching).

game_records = []

for run_dir in sorted(VALID_RUN_DIRS):
    for fpath in sorted(run_dir.glob('trial_*.jsonl')):
        lines = fpath.read_text().splitlines()
        if not lines:
            continue
        events = [json.loads(l) for l in lines if l.strip()]
        start = next((e for e in events if e['event'] == 'trial_start'), None)
        if not start:
            continue
        cond = start['condition']
        if cond.get('with_discussion', False):
            continue

        vote_map = {(e['quest_turn'], e['round']): e['accepted']
                     for e in events if e['event'] == 'team_vote_result'}
        n_quests = sum(1 for e in events if e['event'] == 'quest_result')
        llm_led = sum(
            1 for e in events
            if e['event'] == 'team_proposed'
            and e.get('is_llm_decision')
            and vote_map.get((e['quest_turn'], e['round']), False)
        )

        game_records.append({
            'llm_role': cond['llm_role'],
            'prompt_variant': cond['prompt_variant'],
            'budget': cond['budget'],
            'n_quests': n_quests,
            'llm_led': llm_led,
        })

games_df = pd.DataFrame(game_records)
leader_rate = (games_df
```

```

        .groupby(['llm_role', 'prompt_variant', 'budget'])
        .agg(mean_requests=('n_requests', 'mean'),
              mean_led=('llm_led', 'mean'),
              n_games=('n_requests', 'count'))
        .reset_index())
leader_rate['leader_pct'] = (
    100 * leader_rate['mean_led'] / leader_rate['mean_requests'].replace(0, np.
↳nan)
)

print("Leader-turn stats (no discussion):")
print(leader_rate[['llm_role', 'prompt_variant', 'budget', 'n_games',
                    'mean_requests', 'mean_led', 'leader_pct']].round(2).
↳to_string(index=False))

# Proposal quality (accepted leader proposals, no discussion)
no_disc_acc = accepted_df[~accepted_df['with_discussion']].copy()
pq_all = win_rate_table(
    no_disc_acc.rename(columns={'succeeded': 'llm_win'}),
    ['llm_role', 'prompt_variant', 'budget'])
).sort_values(['llm_role', 'prompt_variant', 'budget'])

COND_LABELS = {
    ('Merlin', 'minimal', 1024): 'M·min·B1k',
    ('Merlin', 'procedural', 1024): 'M·proc·B1k',
    ('Servant', 'minimal', 64): 'S·min·B64',
    ('Servant', 'minimal', 1024): 'S·min·B1k',
    ('Servant', 'procedural', 64): 'S·proc·B64',
    ('Servant', 'procedural', 1024): 'S·proc·B1k',
}
ordered_keys = list(COND_LABELS.keys())
xs = np.arange(len(ordered_keys))

pq_lookup = {(r['llm_role'], r['prompt_variant'], int(r['budget'])): r
              for _, r in pq_all.iterrows()}
lr_lookup = {(r['llm_role'], r['prompt_variant'], int(r['budget'])): r
              for _, r in leader_rate.iterrows()}

fig, axes = plt.subplots(1, 2, figsize=(13, 5.5))
fig.subplots_adjust(top=0.82, wspace=0.38)

# Left - proposal quality (conditional, accepted proposals)
ax = axes[0]
for i, key in enumerate(ordered_keys):
    row = pq_lookup.get(key)
    if row is None:
        continue

```

```

color = ROLE_COLORS[key[0]]
alpha = 0.9 if key[2] == 1024 else 0.45
p, lo, hi = float(row['win_rate']), float(row['ci_lo']), float(row['ci_hi'])
n, wins = int(row['n']), int(row['wins'])

ax.bar(i, p * 100, 0.6, color=color, alpha=alpha, edgecolor='white',
↳linewidth=0.8)
ax.errorbar(i, p * 100,
            yerr=[max(0., (p - lo) * 100)], [max(0., (hi - p) * 100)]],
            fmt='none', color='#333', elinewidth=1.8, capsize=5, zorder=5)
label_y = min(hi * 100 + 2, 104)
ax.text(i, label_y, f"{wins}/{n}", ha='center', va='bottom', fontsize=8.5)

ax.set_xticks(xs)
ax.set_xticklabels([COND_LABELS[k] for k in ordered_keys], rotation=28,
↳ha='right', fontsize=9)
ax.set_ylim(0, 112)
ax.set_ylabel('Proposal quality (%)', fontsize=11)
ax.set_title('P(quest succeeded | LLM led & accepted)\n95% Wilson CI',
↳fontsize=10, fontweight='bold')
ax.axhline(100, color='gray', linestyle=':', linewidth=0.8, alpha=0.5)
ax.legend(handles=[
    mpatches.Patch(facecolor=ROLE_COLORS['Servant'], alpha=0.7,
↳label='Servant'),
    mpatches.Patch(facecolor=ROLE_COLORS['Merlin'], alpha=0.7, label='Merlin'),
    mpatches.Patch(facecolor='gray', alpha=0.9, label='B = 1024'),
    mpatches.Patch(facecolor='gray', alpha=0.45, label='B = 64'),
], fontsize=8.5, loc='lower right')

# Right - leader-turn rate
ax = axes[1]
for i, key in enumerate(ordered_keys):
    row = lr_lookup.get(key)
    if row is None:
        continue
    color = ROLE_COLORS[key[0]]
    alpha = 0.9 if key[2] == 1024 else 0.45
    pct = float(row['leader_pct'])
    led = float(row['mean_led'])
    total = float(row['mean_requests'])

    ax.bar(i, pct, 0.6, color=color, alpha=alpha, edgecolor='white',
↳linewidth=0.8)
    ax.text(i, pct + 0.6, f"{led:.1f}/{total:.1f}",
            ha='center', va='bottom', fontsize=8, color='#333')

ax.set_xticks(xs)

```

```

ax.set_xticklabels([COND_LABELS[k] for k in ordered_keys], rotation=28,
    ↪ha='right', fontsize=9)
ax.axhline(20, color='gray', linestyle='--', linewidth=1.2, alpha=0.7,
    label='1-in-5 uniform (20%)')
ax.set_ylim(0, 35)
ax.set_ylabel('LLM-led quests / total quests (%)', fontsize=11)
ax.set_title('Leader-turn rate\n(annotation: mean led / mean total quests)',
    ↪fontsize=10, fontweight='bold')
ax.legend(fontsize=9, loc='upper right')

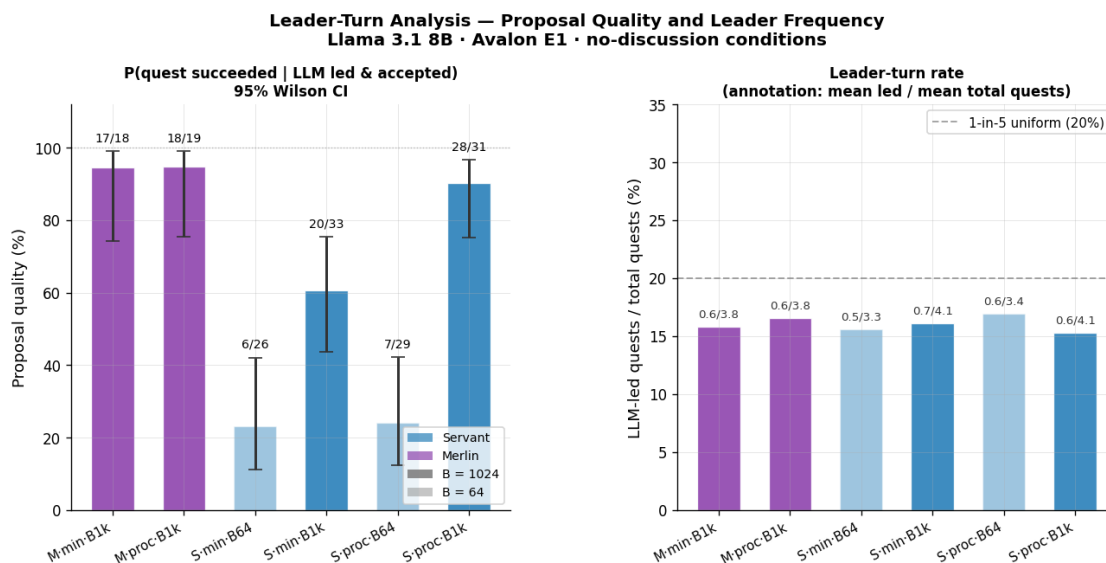
fig.suptitle('Leader-Turn Analysis - Proposal Quality and Leader Frequency\n'
    'Llama 3.1 8B · Avalon E1 · no-discussion conditions',
    fontsize=12, fontweight='bold', y=0.98)

plt.savefig('fig_e1_leader_analysis.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved: fig_e1_leader_analysis.png')

```

Leader-turn stats (no discussion):

llm_role	prompt_variant	budget	n_games	mean_quests	mean_led	leader_pct
Merlin	minimal	1024	30	3.80	0.60	15.79
Merlin	procedural	1024	30	3.83	0.63	16.52
Servant	minimal	64	50	3.34	0.52	15.57
Servant	minimal	1024	50	4.10	0.66	16.10
Servant	procedural	64	50	3.42	0.58	16.96
Servant	procedural	1024	50	4.06	0.62	15.27



Saved: fig_e1_leader_analysis.png

```

[15]: # Figure 9: Proposal quality decomposition - selection effect diagnostics
# Three metrics computed over ALL LLM proposals (no discussion, primary cells):
# (a) Acceptance rate =  $P(\text{accepted} \mid \text{LLM proposed})$  - selection filter
#      ↪ check
# (b) Cond. quality =  $P(\text{succeeded} \mid \text{accepted})$  - current metric
#      ↪ (fixed)
# (c) Uncond. quality =  $P(\text{team is clean} \mid \text{proposed})$  - cleanest measure
# (d) PO self-inclusion =  $P(\text{team includes PO} \mid \text{proposed})$  - confound check
# Red dashed = random-team clean baseline (weighted by actual proposal team
#      ↪ sizes).

def _random_clean_prob(k, n_players=5, n_evil=2):
    n_good = n_players - n_evil
    return comb(n_good, k) / comb(n_players, k) if k <= n_good else 0.0

PRIM_COND_KEYS = [
    ('Servant', 'minimal', 64),
    ('Servant', 'minimal', 1024),
    ('Servant', 'procedural', 64),
    ('Servant', 'procedural', 1024),
    ('Merlin', 'minimal', 1024),
    ('Merlin', 'procedural', 1024),
]
PRIM_LABELS = {
    ('Servant', 'minimal', 64): 'S·min·B64',
    ('Servant', 'minimal', 1024): 'S·min·B1k',
    ('Servant', 'procedural', 64): 'S·proc·B64',
    ('Servant', 'procedural', 1024): 'S·proc·B1k',
    ('Merlin', 'minimal', 1024): 'M·min·B1k',
    ('Merlin', 'procedural', 1024): 'M·proc·B1k',
}

no_disc_prop = prop_df[~prop_df['with_discussion']].copy()

# Random baseline: weighted average over actual team sizes in no-disc proposals
rand_base = no_disc_prop['team_size'].apply(_random_clean_prob).mean()
print(f"Random-team clean baseline (weighted by actual team sizes):
      ↪ {rand_base*100:.1f}%")
print(f" P(clean|size=2) = {_random_clean_prob(2)*100:.0f}%, P(clean|size=3)
      ↪ = {_random_clean_prob(3)*100:.0f}%")

# Compute all four metrics with Wilson CI
metrics = {}
for key in PRIM_COND_KEYS:
    role, variant, budget = key
    sub = no_disc_prop[(no_disc_prop['llm_role']==role) &
                       (no_disc_prop['prompt_variant']==variant) &

```



```

        (no_disc_prop['budget']==budget))

    if sub.empty:
        continue
    n      = len(sub)
    acc_sub = sub[sub['accepted']]
    n_acc  = len(acc_sub)
    suc_w   = int(acc_sub['succeeded'].sum()) if not acc_sub.empty else 0

    metrics[key] = {
        'n':      n,
        'n_acc':  n_acc,
        'acceptance': wilson_ci(int(sub['accepted'].sum()), n),
        'uncond_qual': wilson_ci(int(sub['team_is_clean'].sum()), n),
        'cond_qual':  wilson_ci(suc_w, n_acc) if n_acc > 0 else (np.nan, np.
↳nan, np.nan),
        'p0_inclusion': wilson_ci(int(sub['includes_p0'].sum()), n),
    }

# Plot 4-panel figure
fig, axes = plt.subplots(1, 4, figsize=(17, 5))
fig.subplots_adjust(wspace=0.42, top=0.78)

panel_configs = [
    ('acceptance',    'Acceptance rate\nP(accepted | proposed)',    None),
    ('cond_qual',    'Cond. quality\nP(succeeded | accepted)',    None),
    ('uncond_qual',  'Uncond. quality\nP(clean team | proposed)',  rand_base),
    ('p0_inclusion',  'P0 self-inclusion\nP(P0 in team | proposed)', None),
]

xs      = np.arange(len(PRIM_COND_KEYS))
bar_w = 0.55

for ax, (mkey, title, baseline) in zip(axes, panel_configs):
    for i, key in enumerate(PRIM_COND_KEYS):
        if key not in metrics:
            continue
        p, lo, hi = metrics[key][mkey]
        if np.isnan(p):
            continue
        color = ROLE_COLORS[key[0]]
        alpha = 0.9 if key[2] == 1024 else 0.45
        ax.bar(i, p * 100, bar_w, color=color, alpha=alpha, edgecolor='white',
↳linewidth=0.8)
        ax.errorbar(i, p * 100,
                    yerr=[[max(0., (p - lo) * 100)], [max(0., (hi - p) * 100)]],
                    fmt='none', color='#333', elinewidth=1.5, capsize=4,
↳zorder=5)

```

```

    if baseline is not None:
        ax.axhline(baseline * 100, color='#e74c3c', linestyle='--', linewidth=1.
↪3,
                    alpha=0.85, label=f'Random ({baseline*100:.0f}%)')
        ax.legend(fontsize=8.5, loc='upper left')
        ax.set_xticks(xs)
        ax.set_xticklabels([PRIM_LABELS[k] for k in PRIM_COND_KEYS], rotation=35,
↪ha='right', fontsize=8)
        ax.set_ylim(0, 112)
        ax.set_ylabel('Rate (%)', fontsize=10)
        ax.set_title(title, fontsize=9.5, fontweight='bold')
        ax.axhline(100, color='gray', linestyle=':', linewidth=0.7, alpha=0.4)

fig.suptitle('Proposal Quality Decomposition - Selection Effect Diagnostics\n'
            'Llama 3.1 8B · No-discussion conditions · 95% Wilson CI',
            fontsize=11, fontweight='bold', y=0.97)
fig.legend(handles=[
    mpatches.Patch(facecolor=ROLE_COLORS['Servant'], alpha=0.7,
↪label='Servant'),
    mpatches.Patch(facecolor=ROLE_COLORS['Merlin'], alpha=0.7, label='Merlin'),
    mpatches.Patch(facecolor='gray', alpha=0.9, label='B = 1024'),
    mpatches.Patch(facecolor='gray', alpha=0.45, label='B = 64'),
], loc='upper right', fontsize=8.5, bbox_to_anchor=(0.99, 0.96))

plt.savefig('fig_e1_proposal_decomposition.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved: fig_e1_proposal_decomposition.png')

# Summary table
print('\nProposal quality decomposition:')
print(f'{"Condition":<16} {"N":>4} {"Acc%":>6} {"Cond%":>7} {"Uncond%":>8}
↪{"P0inc%":>7} {"N_acc":>6}')
print('-' * 58)
for key in PRIM_COND_KEYS:
    if key not in metrics:
        continue
    m = metrics[key]
    p_acc = m['acceptance'][0] * 100
    p_con = m['cond_qual'][0] * 100 if not np.isnan(m['cond_qual'][0]) else
↪float('nan')
    p_unc = m['uncond_qual'][0] * 100
    p_p0 = m['p0_inclusion'][0] * 100
    print(f'{PRIM_LABELS[key]:<16} {m["n"]:>4} {p_acc:>6.1f} {p_con:>7.1f}
↪{p_unc:>8.1f} {p_p0:>7.1f} {m["n_acc"]:>6}')

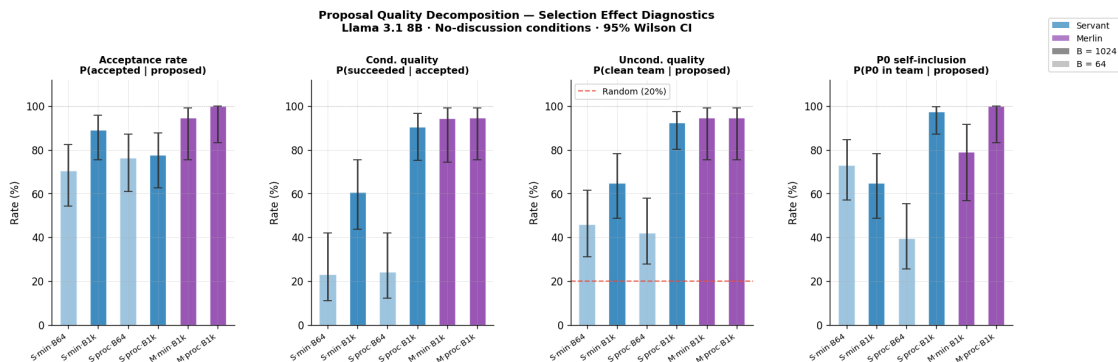
print(f'\nRandom clean baseline: {rand_base*100:.1f}% ')

```

```
f'(size-2: {_random_clean_prob(2)*100:.0f}%, size-3:
↳{_random_clean_prob(3)*100:.0f}%)'
```

Random-team clean baseline (weighted by actual team sizes): 20.1%

$P(\text{clean}|\text{size}=2) = 30\%$, $P(\text{clean}|\text{size}=3) = 10\%$



Saved: fig_e1_proposal_decomposition.png

Proposal quality decomposition:

Condition	N	Acc%	Cond%	Uncond%	P0inc%	N_acc
S.min.B64	37	70.3	23.1	45.9	73.0	26
S.min.B1k	37	89.2	60.6	64.9	64.9	33
S.proc.B64	38	76.3	24.1	42.1	39.5	29
S.proc.B1k	40	77.5	90.3	92.5	97.5	31
M.min.B1k	19	94.7	94.4	94.7	78.9	18
M.proc.B1k	19	100.0	94.7	94.7	100.0	19

Random clean baseline: 20.1% (size-2: 30%, size-3: 10%)